

ARGUS

Autonomous AI QA Agent

Crawls. Plans. Generates. Runs. Triages. Files.

By Everton S. Andrade

QA Automation Engineer / AI-LLM Specialist

Document: August 2026

An MIT-licensed, Vercel-deployable TypeScript project
that rivals and exceeds Cerberus CI in production polish.

1. Overview

Argus is an autonomous AI QA agent that crawls a web application, uses Claude to plan a prioritized test suite, generates 100% deterministic Playwright tests using a template-first approach, executes them, and then uses Claude again to triage failures - distinguishing real bugs from flaky tests, stale selectors, and environment issues.

Key distinction:

Argus is the **GENERATION + TRIAGE** half of the autonomous QA loop. Its sibling project **Cerberus CI** is the **REGRESSION GATE** that locks in existing test suites and prevents regressions on main.

How It Works (7 stages):

1. Ingest	Crawl the app, discover interactive elements, build feature inventory
2. Plan	Claude proposes a prioritized test suite from the feature inventory
3. Codegen	Templates first (30+ deterministic rules), LLM only as fallback
4. Execute	Runs the generated Playwright suite, captures failures with DOM snapshots
5. Triage	Claude classifies each failure: real_bug / flaky / selector_drift / environment
6. File	Creates GitHub Issues with severity labels + inline PR annotations
7. Report	Severity-based CI gate + dashboard data

AI Call Budget: 2 + N per run (1 planning call + 1 triage call per unique failure). Everything else is deterministic and free.

2. Key Features

2.1 Templates First, LLM Second

src/codegen/templates.ts contains 30+ deterministic template rules covering:

- Navigation (navigate-to-route, reload)
- Interaction (click-text, click-selector, check-nth, check-selector, fill, select)
- Assertions (assert-text, assert-checked, assert-not-checked, assert-should-see, assert-should-not-see)
- Security (require-https, reject-empty-input, validate-email-format)
- Accessibility (accessible-name, keyboard-navigable, no-autoplay-media, color-contrast)

The LLM is only called if no template matches - and in 100% of demo runs, templating covered everything.

2.2 Verdict Cache

Error signatures are normalized (strip line:col, UUIDs, timestamps, URLs) and hashed (SHA-256, 16-char hex):

Example:

```
"AssertionError at src/page.ts:42:17 in https://api.com/u/550e8400"  
-> "AssertionError at in api.com u"  
-> SHA-256 -> 16-char hex key
```

30-day TTL per entry, cached in data/runs/triage-cache.json. Repeated failures skip the AI call entirely - up to 60% cost reduction on apps with flaky tests.

2.3 Severity-Based Gate

The CI gate is NOT pass/fail - flakiness never blocks merges:

```
CI FAILS only if: new real bugs at or above severityFailThreshold  
CI PASSES if: all failures are flaky / selector_drift / environment  
OR all real bugs are duplicates of existing issues
```

3. Architecture

3.1 Project Structure

```
argus/  
+-- src/  
| +-- cli/ # CLI entry, pipeline, CI reporting  
| +-- shared/ # Types, config, AI provider abstraction  
| +-- ingestion/ # Crawl web app, build feature inventory  
| +-- planner/ # AI stage: generate test plan  
| +-- codegen/ # Deterministic codegen (0 AI calls)  
| +-- execution/ # Run Playwright, capture failures  
| +-- triage/ # AI classification + verdict cache  
| +-- bug-filer/ # GitHub Issues filing + dedupe  
+-- dashboard/ # Next.js 15 + Tailwind dashboard  
+-- demo-app/ # Demo app with known bugs  
+-- test/ # 313 tests (all fixture-based)  
+-- docs/ # Demo assets, interview talking points
```

3.2 Provider Abstraction

The AIProvider interface abstracts LLM calls, mirroring Cerberus's pluggable adapter design. Includes ClaudeProvider, OpenAICompatibleProvider, MockProvider, and OllamaAdapter. Config-driven via ARGUS_AI_PROVIDER env var.

3.3 Running Argus

Quick Start (No API Key Required):

```
git clone https://github.com/EvertonSt/argus  
cd argus  
npm install  
npm test # 313 tests, no API key needed  
npm run run:mock # full pipeline demo with mocked AI
```

Run Against Your App:

```
npx argus run --target http://localhost:3000
```

4. Configuration

All config is env-driven:

ARGUS_AI_PROVIDER	Provider: claude, openai, ollama, mock
ARGUS_AI_MODEL	Model: sonnet4, gpt4o, llama3.3, gpt4o-mini
ARGUS_TARGET_HOST	Target host: 4317 to test
ARGUS_GIT_HUB_TOKEN	GitHub token (absent -> dry-run)
ARGUS_GIT_HUB_REPO	Hub repo: owner/repo for Issues
ARGUS_GATE_SEVERITY	Min severity for CI gate failure
ARGUS_MAX_AI_CALLS	Max AI call cap
ARGUS_BROWSER	Browser: playwright
ARGUS_OPENAI_BASE_URL	OpenAI Compatible providers
ARGUS_OPENAI_MODEL	Model name for OpenAI provider

5. OpenAI Compatible Endpoints

Argus supports any OpenAI-compatible API endpoint, including:

- OpenAI (default)
- DeepSeek
- Gemini (via OpenAI-compatible proxy)
- Ollama (with OpenAI-compatible mode enabled)
- Any local/custom LLM serving OpenAI-format API

Example: Use DeepSeek

```
export ARGUS_AI_PROVIDER=openai
export ARGUS_OPENAI_BASE_URL=https://api.deepseek.com/v1
export ARGUS_OPENAI_API_KEY=sk-your-deepseek-key
export ARGUS_OPENAI_MODEL=deepseek-chat
npm run run:mock
```

6. Security & Hardening

- **Dashboard server:** path allowlist, blocks dotfiles/secrets/imports/node_modules
- **Network binding:** 127.0.0.1 by default; opt-in --host 0.0.0.0
- **Dedupe:** signature-based (title + featureId + errorClass + verdict), threshold 0.55
- **No auto-fix:** even 94%-confidence results require human approval
- **Tokenless dry-run:** GitHub Issues filing auto-dry-runs if token missing

7. Portfolio Context

Project	Role	Focus	Tests
Argus	Solo founder	Generation + Triage	313
Cerberus CI	Solo founder	Regression gate	237

Argus creates the test suite - it's the autonomous agent that discovers, plans, generates, and triages. Cerberus CI gates existing suites - it's the deterministic CI gate that prevents regressions. Together they form a complete autonomous QA loop: generate -> triage -> gate -> repeat.

For full details, source code, and live demos:

<https://github.com/EvertonSt/argus>